

# Detection Gaps for Spatial Memory Corruption in LLM-Generated C: An Intra-Object Overflow Study Across Detectors, Toolchains, and Code Models

Sadik Erisen

elmanoukdesign@gmail.com

## Abstract

Large language models are increasingly used to generate C code that is compiled and run inside sandboxes we trust to contain untrusted behaviour. That trust rests on a stack of commodity defences: compiler warnings, static analyzers, build-time hardening, and sanitizers. We ask a narrow, falsifiable question: for the specific class of spatial memory corruption, which of these defences actually fire, and does the code a model produces land in the gaps they leave?

We run three experiments. **H1** points ten independent commodity detectors at six memory-corruption classes and records which are caught. One class — *intra-object* (adjacent-field) overflow, an unbounded write that overwrites a neighbouring struct field but stays inside a single valid allocation — is missed by *every* detector, including AddressSanitizer and Valgrind, because no allocation boundary is crossed. **H2** shows this gap is not a toolchain artefact: it holds across five compilers (gcc-11, gcc-12, clang-14, clang-15, tcc), two `FORTIFY_SOURCE` levels, and AddressSanitizer under both gcc and clang, and across two CPU architectures. **H3** asks whether real code models produce this class. We give three local code models 216 security-neutral tasks, each with a fixed-size struct field, and check every generated snippet against a single-translation-unit, AddressSanitizer-backed ground truth calibrated on golden-safe and golden-vulnerable references. 11 of 213 usable snippets contain a confirmed spatial overflow (a lower bound), concentrated almost entirely in the smallest model (10/70 (14.3%, 95% CI 8–24%) versus 0–1.4% for the 7B models). The commodity static stack (gcc `-Wall`, `-fanalyzer`, cppcheck) diagnosed the overflow in 0 of the 11: it is structurally blind to input-length-dependent overflow in a function it analyzes in isolation. We connect the three results at the detection-layer level rather than by asserting more than we measured: the shipped *static* layer catches none of these generated overflows, and H1/H2 show the shipped *runtime* tool (AddressSanitizer) is blind to the intra-object subclass, so neither layer is a sufficient detector for this class. One generated overflow (`parse_kv`) writes *through* an adjacent struct field before reaching the allocation boundary; had the model bounded the loop, the same write would be an intra-object overflow invisible to ASan — the H1 probe in generated form. We report every number with its limits, including why the rate is a lower bound and why a hand-verification pass removed one oracle false positive.

## 1 Introduction

When an LLM-driven agent compiles and runs its own C code inside a sandbox, the sandbox has to hold in two places at once: at the container boundary, and at the memory boundary inside the process. This paper is only about the second. We do not argue about whether an agent is “capable” of writing an exploit; we measure which memory-safety weaknesses survive the commodity tooling we normally trust, and whether generated code falls into the surviving classes.

The contribution is three linked, falsifiable measurements rather than a new tool:

- **H1 (detector coverage).** Across ten commodity detectors and six memory-corruption classes, intra-object overflow is a *total* detection gap — missed by every static analyzer, every compiler warning set, build-time hardening, and both major sanitizers. We define the operational states (silent, detected, crashed, prevented, clean) precisely so “missed” is not a matter of opinion.
- **H2 (toolchain generality).** The gap is structural, not a property of one compiler version: it survives five compilers, two hardening levels, both sanitizer back-ends, and two architectures (aarch64 and x86\_64).
- **H3 (does generated code hit the layer that is blind).** Three local code models, given fixed-buffer tasks with no security framing, produce confirmed spatial overflows at a measurable rate — concentrated in the smallest model — and the shipped static layer diagnosed 0 of them. Ground truth is automated, calibrated, and then hand-verified; the verification pass removed one oracle false positive and confirmed the numbers rest on AddressSanitizer, not on inspection.

We are deliberate about what each number is and is not. The H3 vulnerable rate is a lower bound; it is conditional on tasks that involve a fixed-size buffer; and it is measured on three small open-weight models on one machine. Section 6 states every such limit rather than burying it.

## 2 Threat model and definitions

### 2.1 What “the sandbox” means here

By sandbox we mean the stack normally trusted to hold the line around compiled-and-executed code: the compiler and its hardening flags, the static analyzers run in CI, and the sanitizers used in testing. We treat each as an independent *detector* and ask, per memory-corruption class, whether it fires.

### 2.2 Operational states of a run

For a build that exercises a bug on a given input, we classify the outcome by direct evidence (the process’s exit signal, the diagnostic it emits, and a self-check the probe prints):

#### **silent**

Corruption confirmed by the probe self-check, the process exits 0, and no defence emitted any diagnostic. Exploitable and undetected.

#### **detected**

A defence named the memory error and stopped the process (`_FORTIFY_SOURCE __chk`, stack canary, a sanitizer report).

#### **crashed**

A raw `SIGSEGV/SIGBUS/SIGABRT` with no diagnostic: the corruption faulted (a denial-of-service outcome), but nothing explained it.

#### **prevented**

The unsafe operation was refused before corruption (an allocator returned `NULL` or aborted at the call), and the self-check reports clean.

## clean

The bug did not take effect on this input.

The distinction that matters for this paper is **silent** versus everything else. A **silent** outcome is the definition of a detection gap: real corruption, zero signal.

## 2.3 Intra-object versus boundary overflow

An *allocation-boundary* overflow writes past the end of a heap or stack object; red-zone tools (AddressSanitizer) and shadow-memory tools (Valgrind) see it because the write crosses into instrumented memory. An *intra-object* overflow writes past one struct field into an adjacent field of the *same* allocation:

```
struct rec { char name[16]; int uid; }; /* one allocation */
strcpy(r->name, attacker_input); /* 20 bytes: name[16] overrun spills into uid */
```

No allocation boundary is crossed, so boundary-based tools stay silent by construction. This is the class the paper is built around.

## 3 H1: detector coverage of memory-corruption classes

### 3.1 Method

We point ten independent commodity detectors at six memory-corruption classes and record CAUGHT versus MISSED, saving raw per-cell evidence so every verdict is auditable. The detectors are, static/compile-time: gcc -Wall -Wextra, gcc -fanalyzer, gcc LTO type-mismatch, clang -Warray-bounds, clang --analyze, cppcheck; and runtime: \_FORTIFY\_SOURCE=2, AddressSanitizer, UndefinedBehaviorSanitizer, Valgrind. Each runtime column is built with isolation flags so it measures exactly one tool (canary and FORTIFY disabled when measuring a sanitizer, etc.). A compile diagnostic counts only if a memory-safety checker fires on a real line; meta-noise (“unknown warning option”) is filtered and never counts as a catch.

The six classes are ABI/type skew (CWE-843), out-of-bounds write in three variants (intra-object, heap, stack; CWE-787), use-after-free (CWE-416), and integer-overflow allocation (CWE-190).

### 3.2 Results

Table 1 gives the coverage. The headline: intra-object overflow is caught by *no* detector — a total gap — while the heap and stack variants of the same CWE-787 are caught at runtime by ASan and Valgrind. Static analysis (all six compile-time detectors) caught none of the six classes on these probes. Under the shipped build (-O2 -D\_FORTIFY\_SOURCE=2, default canary), five of six classes are **silent** and only the stack overflow is **detected** (by the canary).

The result is not that these tools are weak in general — ASan and Valgrind catch the heap and stack overflows cleanly. It is that the *intra-object* variant is invisible to all of them at once, and that the shipped hardening bundle leaves five of six classes silent. The intra-object row is the load-bearing cell of this paper and is re-verified in Section 4 across five compilers and two architectures; the integer-overflow and ABI-skew rows summarise the recorded detector matrix and mark those partial/absent cells conservatively.

Table 1: H1: which commodity detector catches which class. “static” = all six compile-time detectors. Shipped state is the `-O2 -D_FORTIFY_SOURCE=2` build.

| class                                   | static | FORTIFY | ASan        | Valgrind    | shipped state     |
|-----------------------------------------|--------|---------|-------------|-------------|-------------------|
| <code>oob_intra</code> (adjacent field) | miss   | miss    | <b>miss</b> | <b>miss</b> | silent            |
| <code>oob_heap</code>                   | miss   | miss    | CAUGHT      | CAUGHT      | silent            |
| <code>oob_stack</code>                  | miss   | miss    | CAUGHT      | CAUGHT      | detected (canary) |
| <code>use-after-free</code>             | miss   | –       | CAUGHT      | CAUGHT      | silent            |
| <code>int-overflow alloc</code>         | miss   | –       | partial     | partial     | silent            |
| <code>abi/type skew</code>              | miss   | –       | –           | –           | silent            |

## 4 H2: the gap is a structural property, not a toolchain artefact

### 4.1 Method

We re-run the six classes through five compilers (`gcc-11`, `gcc-12`, `clang-14`, `clang-15`, `tcc`) for compile-time warnings, through `_FORTIFY_SOURCE` levels 2 and 3 for the shipped state, and through AddressSanitizer built by both `gcc-12` and `clang-15`. H2 holds if intra-object stays a total gap and no compiler or level newly catches it.

### 4.2 Results

No compiler warned on intra-object at any level; the shipped state is silent under `_FORTIFY_SOURCE=2` and `=3`; and ASan misses it under both `gcc` and `clang`. The gap holds across every toolchain tested. Two honest side-findings: `gcc-12` is stricter than `gcc-11` (it emits a compile-time use-after-free warning where `gcc-11` is silent), and the five-of-six-silent shipped pattern is stable across all compilers. Because these experiments ran on `x86_64` while the companion sandbox study ran on `aarch64` with `gcc-15`, the intra-object gap is confirmed across two architectures and a wide `gcc` version range (11, 12, 15). The result is consistent with its cause: a boundary-based tool cannot see a write that never crosses a boundary, regardless of compiler.

## 5 H3: do real code models produce this class, and does the static stack miss it?

### 5.1 Method

We test three local open-weight code models — `qwen2.5-coder:7b`, `deepseek-coder:6.7b`, and `qwen2.5-coder:1.5b` — on 18 security-neutral tasks. Each task asks for a struct with a fixed-size `char` field and a function that fills it from variable-length input; no task mentions safety, overflow, or bounds. We draw four samples per task per model (one at temperature 0, three at 0.6), for 216 generations.

**Provided contract.** To let an automated oracle bind to the output, each prompt specifies the exact struct and function signature (`void fn(struct S *s, const char *input)`). This fixes the interface; it does *not* hint at safety. What we measure is whether, given that interface, the model bounds the copy.

**Two-oracle ground truth.** Every snippet is judged by an automated oracle, not by inspection. The driver is concatenated into the *same translation unit* as the snippet, so it binds to the model’s own struct and function; any signature or layout deviation becomes a compile error we count as

`compile_fail` rather than silent undefined behaviour. Under AddressSanitizer we sweep input lengths  $\{N-1, N, N+1, N+8, N+128\}$  for a field of size  $N$  and apply two oracles:

**Oracle A (ASan).**

A grossly oversized input that overruns the whole allocation triggers a heap-buffer-overflow report.

**Oracle B (intra-object).**

The trailing scalar field is set to a sentinel; an off-by-one or small overflow that stays *inside* the allocation clobbers the sentinel — the exact class Oracle A (and ASan) cannot see. Oracle B is applied only to tasks with a trailing scalar and a field size that is a multiple of the scalar’s alignment (no padding between field and sentinel).

A snippet is vulnerable if either oracle fires.

**Calibration.** Before any generated code is judged, every one of the 18 drivers is validated against a golden-safe reference (`strncpy + explicit null`) and a golden-vulnerable reference (`strcpy`); the driver is trusted only if it reports the safe reference clean and the vulnerable reference flagged. All 18 drivers passed.

**Static-stack classification.** Independently, each snippet is run through the commodity static stack (`gcc -Wall -Wextra`, `gcc -fanalyzer`, `cppcheck`) on the snippet alone. The money metric is: of the oracle-confirmed vulnerable snippets, how many did the static stack flag?

## 5.2 Results

Of 216 generations, 213 compiled against the oracle; 3 were `compile_fail` (contract deviations: one corrupted generation, one write to a `const` pointer, one `typedef`’d struct that left `struct header` undefined). We exclude all three from the denominator; on inspection the `typedef` case contains an unbounded copy and would likely be vulnerable, so excluding it keeps the rate a lower bound.

11 of 213 usable snippets contain an AddressSanitizer-confirmed spatial overflow, an overall rate of 5.2% (95% CI 3%–9%). The result is dominated by model size (Table 2): the smallest model, `qwen2.5-coder:1.5b`, produced the class in 10/70 (14.3%, 95% CI 8–24%), while `deepseek-coder:6.7b` produced 1/71 (1.4%) and `qwen2.5-coder:7b` produced 0/72 (0%). We state this as an observation on three points along the model-size axis, not a scaling law; the intervals are wide.

Table 2: H3: confirmed spatial overflow rate by model (usable snippets), and how the commodity static stack diagnosed the confirmed cases.

| model                                                              | confirmed / usable | rate  | 95% CI |
|--------------------------------------------------------------------|--------------------|-------|--------|
| qwen2.5-coder:1.5b                                                 | 10/70              | 14.3% | 8–24%  |
| deepseek-coder:6.7b                                                | 1/71               | 1.4%  | 0–8%   |
| qwen2.5-coder:7b                                                   | 0/72               | 0.0%  | 0–5%   |
| <i>static-stack diagnosis of the 11 confirmed overflows:</i>       |                    |       |        |
| overflow diagnosed by any static tool: 0/11                        |                    |       |        |
| zero static diagnostics at all: 8/11                               |                    |       |        |
| unrelated warning on the function (truncation, sign-compare): 3/11 |                    |       |        |

**The static layer is blind to this class.** No member of the commodity static stack diagnosed the actual overflow in *any* of the 11 confirmed cases (0/11). 8 drew no diagnostic at all; the remaining 3 drew a warning on the function that points at a *different* defect (a `-Wstringop-truncation` about missing null-termination, two `-Wsign-compare`) but not at the overflow AddressSanitizer confirmed. This is not a claim that the tools are weak in general — gcc’s object-size checks fire when a size is statically known. It is that these overflows are input-length-dependent writes in a function the analyzer sees in isolation, where the caller’s input length is opaque; that is precisely the real-world condition of a parser compiled in its own translation unit, and it is undecidable at compile time. Only runtime AddressSanitizer caught the 11, and ASan is not part of a shipped build.

**Where this meets H1/H2.** We do not claim generated code produced an ASan-invisible intra-object overflow: in this sample it did not (0 such cases; the one candidate our intra-object oracle flagged was a false positive — a function that legitimately assigns its trailing field — removed by hand-verification). The connection is at the layer level. The shipped *static* layer caught none of the 11 generated overflows (this section); H1/H2 show the shipped-adjacent *runtime* tool is blind to the intra-object subclass. Neither layer is a sufficient detector for the class. One case makes the boundary concrete: `qwen2.5-coder:1.5b`’s `parse_kv` copies into `key[16]` with an unbounded loop; its ASan report shows the write reaching “0 bytes to the right of [the] 64-byte region,” i.e. it passes *through* the adjacent `val[48]` field (offsets 16–63 of the 64-byte struct) and faults only at the allocation boundary. Had the model bounded that loop to the struct — as other samples did — the same `key`→`val` overwrite would be an intra-object write that never crosses a boundary and that AddressSanitizer would not report: the H1 probe, arrived at from generated code.

**The two-oracle design and what it contributed.** Our ground truth combined an allocation-boundary oracle (A, AddressSanitizer) with an intra-object sentinel oracle (B). In this run *all 11 confirmed findings came from oracle A*; oracle B produced no true positives and one false positive (the legitimate trailing-field assignment above), which hand-verification caught. We report this plainly: the rate rests on ASan. Oracle B’s value here was as a check for the ASan-invisible subclass that, in this sample, did not occur.

## 6 Threats to validity

- **The H3 rate is a lower bound.** Oracle B cannot see a mid-struct `field[i]`→`field[i+1]` overflow, the multi-`char`-field tasks are judged by Oracle A alone, and one of the three excluded `compile_fails` contains an unbounded copy that would likely be vulnerable if it compiled. Every one of these effects can only raise the true rate above what we report.
- **Oracle B produced a false positive, and the rate rests on ASan.** The intra-object sentinel oracle assumes the function never writes its trailing scalar; a generated function that legitimately assigns that field (`s->count = 1`) trips the sentinel without any overflow. Our golden-safe calibration reference did not write the trailing field, so calibration passed without exercising this case. We caught the single occurrence by hand-verification and excluded it; all 11 confirmed findings rest on AddressSanitizer (oracle A), not on oracle B. A corrected oracle B would compare the clobbered bytes against a partial-overflow signature (high bytes of the sentinel intact) rather than exact inequality.

- **Conditional on fixed-buffer tasks.** The tasks were chosen to involve a fixed-size buffer, so the rate is “conditional on a fixed-buffer task,” not a rate over all code a model writes.
- **Three small open models, one machine.** We test three open-weight models in the 1.5–7B range locally; we do not claim these rates transfer to frontier hosted models. The provided-contract design fixes the interface and may differ from free-form generation.
- **Standard tool configuration.** A non-default ASan build with intra-object red zones (`-fsanitize-address-field-padding`) can catch some intra-object overflows, but it is C++-oriented, opt-in, and part of no shipped build; the practical gap stands. We state this rather than omit it.
- **Compile-time versus runtime.** The static-stack-miss metric is a *compile-time* claim and is kept distinct from the shipped-`_FORTIFY_SOURCE` runtime state; we do not blur a static miss into a runtime catch.
- **Single toolchain per version.** One gcc/clang/valgrind build each. The intra-object result is structural and H2 confirms it across two architectures, but the broader matrix should be re-run elsewhere to confirm.

## 7 Related work

AddressSanitizer [1] and Valgrind/Memcheck [2] are the red-zone and shadow-memory tools whose boundary model is precisely why intra-object overflow is invisible to them. The closest prior art on the intra-object case is ASan’s own field-padding extension [4], which inserts poisoned padding between fields to catch adjacent-field overflow — but only for C++ non-standard-layout classes, behind the experimental `-fsanitize-address-field-padding` flag, and in no shipped build; that scoping is exactly why the practical gap we measure stands. `_FORTIFY_SOURCE` [3] provides the build-time `__builtin_object_size` checks we measure at the shipped-hardening layer. A separate line of work reports that code models generate insecure code — Pearce et al. [5] found roughly 40% of Copilot completions across 89 security-relevant scenarios were vulnerable. Our contribution is not that claim but its connection to a measured, toolchain-general detector gap: we show generated code lands in a spatial-overflow class the shipped detection stack cannot see, and we separate the static layer (blind to input-dependent overflow in isolated functions) from the runtime layer (blind to the intra-object subclass).

## 8 Conclusion

Spatial memory corruption has a specific, reproducible blind spot — intra-object overflow — that no commodity detector catches, and that survives five compilers, two hardening levels, both sanitizer back-ends, and two architectures. Independently, small code models produce spatial overflows on ordinary fixed-buffer tasks at a measurable rate (dominated by the smallest model), and the shipped static layer diagnosed none of them. Read together at the layer level: the static layer misses input-length-dependent overflow in isolated functions, and the runtime layer (AddressSanitizer) misses the intra-object subclass, so a pipeline that compiles and runs LLM-generated C cannot rely on compiler warnings, static analysis, build hardening, *or* AddressSanitizer to be a sufficient detector for this class. Detecting adjacent-field overflow needs an oracle that reasons about struct-field boundaries, not allocation boundaries — and that oracle is not in the default stack today.

**Artifacts.** The detector matrix, toolchain sweep, H3 generation-and-oracle harness, all generated snippets, the calibration references, and raw per-cell evidence are released with the paper.

## References

- [1] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov. “AddressSanitizer: A Fast Address Sanity Checker.” *2012 USENIX Annual Technical Conference (USENIX ATC '12)*, pp. 309–318, 2012.
- [2] N. Nethercote and J. Seward. “Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation.” *Proc. 28th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI '07)*, pp. 89–100, 2007.
- [3] J. Jelinek. “Object size checking to prevent (some) buffer overflows.” GCC patches mailing list, Sept. 2004. <https://gcc.gnu.org/legacy-ml/gcc-patches/2004-09/msg02055.html>. (Origin of `_FORTIFY_SOURCE`; see also the glibc manual.)
- [4] Google Sanitizers project. “AddressSanitizerIntraObjectOverflow.” <https://github.com/google/sanitizers/wiki/AddressSanitizerIntraObjectOverflow>. (Experimental `-fsanitize-address-field-padding`; C++ non-standard-layout classes.)
- [5] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri. “Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions.” *2022 IEEE Symposium on Security and Privacy (SP)*, pp. 754–768, 2022.